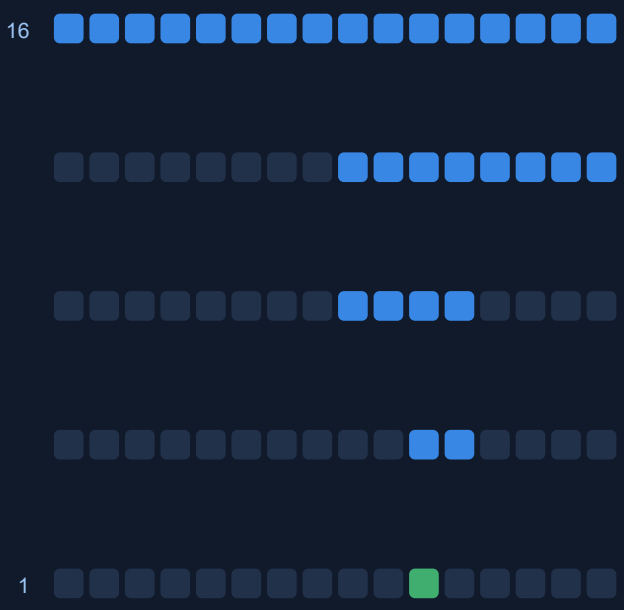
АЛГОРИТМЫН ҮНДЭС · PYTHON · ХАЙЛТ 2/2

Binary search

Хоёртын хайлт: бичих, мөрдөх, алдаагүй хэрэгжүүлэх



Түвшин: Хэрэгжүүлэх | **Жишээ:** Python 3 | **Урьдчилсан мэдлэг:** Linear search, while, Big O

Энэ хичээлийг уншсаны дараа та:

- Binary search-ийг цээжлэлгүйгээр, **учир шалтгаанаас нь** гаргаж бичнэ: `lo`, `hi`, `mid`, `+1`, `-1` яагаад гэдгийг тайлбарлана.
- Кодоо цаасан дээр **мөрдөж** (trace), эргэлт бүрийн хувьсагчдыг хүснэгтээр гаргана.
- Binary search-ийн **5 түгээмэл алдааг** таньж засна, кодоо **тестээр** баталгаажуулна.
- Давхардал, оруулах байр зэрэг хувилбаруудыг бичиж, Python-ий `bisect` модулийг хэрэглэнэ.
- Linear ба binary search-ийн аль нэгийг **хэзээ** сонгохоо мэднэ.

Агуулга

01 Санаа
02 Алхам алхмаар бичих
03 Гараар мөрдөх (trace)
04 Түгээмэл алдаа ба засвар
05 Давхардал, оруулах байр, bisect
06 Тестлэх
07 Linear search-тэй харьцуулах
08 Дасгал
09 Хураангуй

> **Нэг өгүүлбэрээр**
> **Binary search** (хоёртын хайлт): жагсаалт **эрэмбэлэгдсэн** бол дундахыг нь хар, хэрэггүй хагасыг нь хая, үлдсэн хагаст давт.

01 Санаа

> **Өмнөх хичээлээс (Хайлт 1/2: Linear search)**
> Хайлтын функц `items`, `target` авч, олдвол **индексийг**, олдохгүй бол `-1`-ийг буцаана. Linear search элемент бүрийг шалгадаг тул **O(n)**: сая элементэд сая хүртэл харьцуулалт. Binary search ижил гэрээтэй, гэхдээ хамаагүй хурдан.

Тоглоом: найз чинь 1-ээс 100-ын хооронд тоо бодсон. Чи таах бүрт тэр «их», «бага» эсвэл «зөв» гэж хариулна. Хамгийн ухаалаг стратеги бол үргэлж **дундах** тоог хэлэх. Бодсон тоо 37 бол:

Таалт	Боломжит хүрээ	Хэлсэн тоо	Хариу	Шинэ хүрээ
1	1 … 100	50	«их»	1 … 49
2	1 … 49	25	«бага»	26 … 49
3	26 … 49	37	**«зөв»**	—

Таалт бүр боломжит хүрээг **хагасаар** багасгаж байна. 100 тооноос дээд тал нь 7 таалтаар заавал олно. Нэг нэгээр нь тааж эхэлбэл 100 хүртэл таалт хэрэгтэй.

> **Заавал байх нөхцөл: өгөгдөл эрэмбэлэгдсэн байх**
> 50 «их» гэвэл бодсон тоо 50-аас **доош** гэдгийг бид мэднэ — тиймээс 51…100-г шалгалгүйгээр хаяж чадна. Энэ нь тоонууд **дараалаараа** байгаа учраас л боломжтой. Эрэмбэгүй жагсаалтын дунд элемент бусад элементийн талаар юу ч хэлж өгөхгүй.

Гурван хувьсагч

Жагсаалт дээр «боломжит хүрээ»-г хоёр индексээр хадгална:

Хувьсагч	Утга	Эхлэх утга
`lo`	Хүрээний зүүн хил (энэ индекс хүрээнд **багтана**)	`0`
`hi`	Хүрээний баруун хил (энэ индекс ч **багтана**)	`len(items) - 1`
`mid`	Хүрээний дунд — одоо шалгах элемент	`(lo + hi) // 2`

> **Гол санаа (invariant — үргэлж үнэн байх зүйл)**
> Хэрэв target жагсаалтад байгаа бол тэр заавал `items[lo]` … `items[hi]` хооронд байна. Эргэлт бүр энэ хүрээг хагас болгоно. Хүрээ хоосон болбол (`lo > hi`) — target **байхгүй**.

Хэдэн алхам хийх вэ?

n элементийг хагасалсаар 1 болгоход $\log_2 n$ орчим алхам хэрэгтэй. Яг тоо нь $\log_2 n$-ийн **бүхэл хэсэг + 1**:

n (элементийн тоо)	Linear: хамгийн ихдээ	Binary: хамгийн ихдээ
10	10	4
100	100	7
1 000	1 000	10
1 000 000	1 000 000	**20**
1 000 000 000	1 000 000 000	**30**

Элементийн тоо **1000 дахин** өсөхөд binary search ердөө **10 алхмаар** нэмэгдэж байна.

02 Алхам алхмаар бичих

Кодыг цээжлэх шаардлагагүй — 1-р бүлгийн санаанаас мөр бүрийг гаргаж авъя.

Алхам 1. Хүрээг бүх жагсаалтаар эхлүүл: `lo, hi = 0, len(items) - 1` — `hi` бол сүүлийн **индекс**, урт биш.

Алхам 2. Хүрээ хоосон биш байх хооронд давт: `while lo <= hi:` — `lo == hi` үед 1 элемент үлдсэн, түүнийг ч шалгах ёстой тул `<=`.

Алхам 3. Дундыг ол: `mid = (lo + hi) // 2` — `//` бол бүхэл хуваалт, индекс бүхэл тоо байх ёстой.

Алхам 4. Гурван тохиолдлыг шалга:

Нөхцөл	Утга нь	Хийх зүйл
`items[mid] == target`	олдлоо	`return mid`
`items[mid] < target`	target **баруун** талд байна	`lo = mid + 1`
`items[mid] > target`	target **зүүн** талд байна	`hi = mid - 1`

Алхам 5. Давталтаас гарвал хүрээ хоосон болсон гэсэн үг: `return -1`.

```python
def binary_search(items, target):
    lo, hi = 0, len(items) - 1          # 1. хүрээ = бүх жагсаалт
    while lo <= hi:                     # 2. хүрээ хоосон биш байхад
        mid = (lo + hi) // 2            # 3. дунд индекс
        if items[mid] == target:        # 4а. олдлоо
            return mid
        elif items[mid] < target:       # 4б. баруун талд байна
            lo = mid + 1
        else:                           # 4в. зүүн талд байна
            hi = mid - 1
    return -1                           # 5. хүрээ хоосон → байхгүй

nums = [2, 5, 8, 12, 16, 23, 38, 41, 56, 61, 67, 72, 80, 85, 91, 99]
print(binary_search(nums, 67))          # 10
print(binary_search(nums, 40))          # -1
```

> **Яагаад mid + 1, mid − 1 гэж?**
> `items[mid]`-ийг бид дөнгөж шалгаад тараагүйг мэдсэн. Тиймээс түүнийг дараагийн хүрээнд дахин оруулах шаардлагагүй. Энэ ±1 нь хүрээг эргэлт бүрт **заавал** багасгадаг — эс тэгвээс давталт мөнхөрч болно (4-р бүлгийн 2-р алдаа).

Рекурсив хувилбар

Ижил санааг рекурсээр бичиж болно: `lo`, `hi` нь параметр болж, «хүрээ хоосон» нь суурь тохиолдол болно:

```python
def binary_search_rec(items, target, lo=0, hi=None):
    if hi is None:
        hi = len(items) - 1             # анхны дуудлага: бүх жагсаалт
    if lo > hi:                         # суурь тохиолдол: хүрээ хоосон
        return -1
    mid = (lo + hi) // 2
    if items[mid] == target:
        return mid
    if items[mid] < target:
        return binary_search_rec(items, target, mid + 1, hi)
    return binary_search_rec(items, target, lo, mid - 1)
```

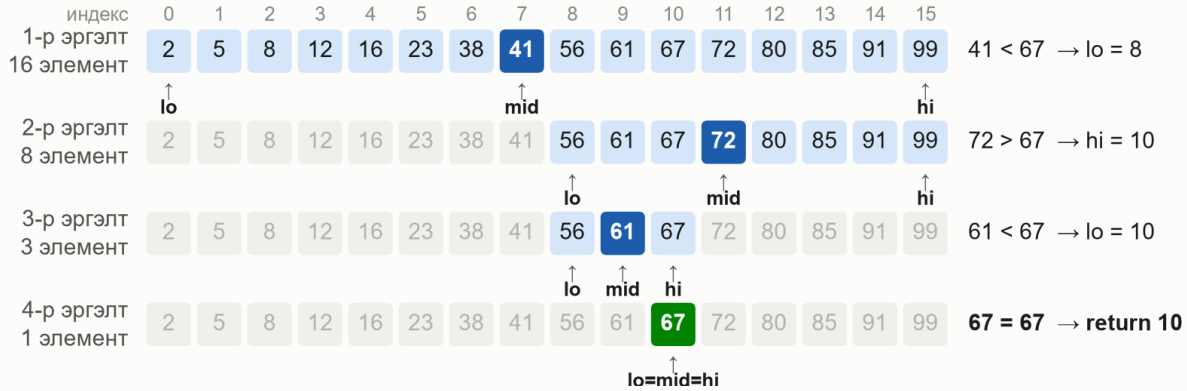
Давталттай хувилбар O(1) санах ой ашигладаг бол рекурсив хувилбар дуудлагын стек дээр **O(log n)** зай эзэлнэ (сая элементэд ердөө 20 давхар — асуудалгүй).

> **Зүсэлт (slice) хэрэглэж болохгүй**
> `binary_search_rec(items[mid + 1:], target)` гэж бичвэл: (1) зүсэлт бүр шинэ хуулбар үүсгэдэг тул нийт **O(n)** болж binary search-ийн бүх ашиг алга болно; (2) шинэ жагсаалтын индекс 0-ээс эхэлдэг тул буцаасан индекс **буруу** болно. Тиймээс `lo`, `hi` индексийг дамжуулдаг.

03 Гараар мөрдөх (trace)

Код бичсэний дараах хамгийн сайн шалгалт бол цаасан дээр **мөрдөх**: хүснэгт зурж, давталтын эргэлт бүрт хувьсагчдын утгыг бичнэ. Хоёр тохиолдлыг мөрдье — олдох ба олдохгүй.

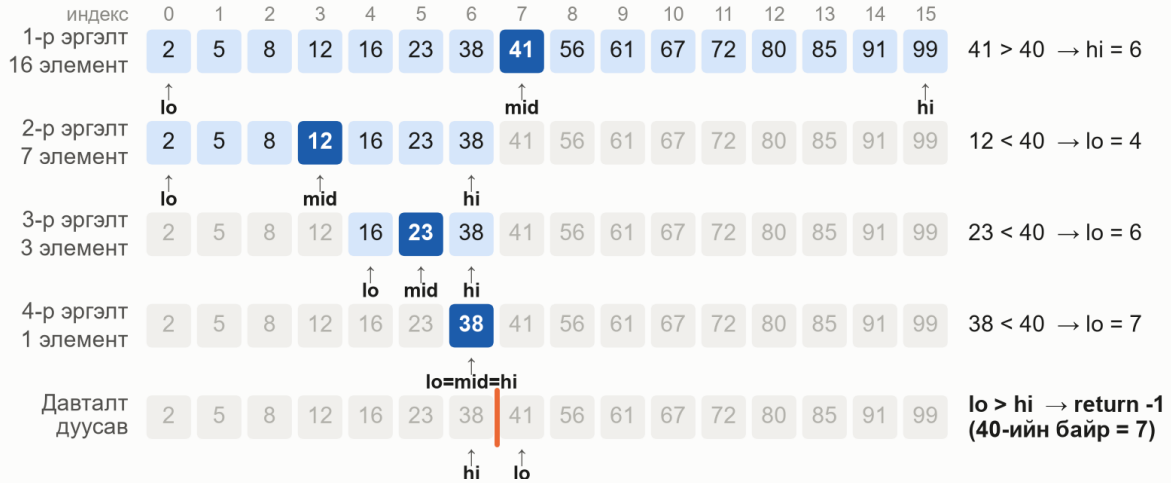
Тохиолдол 1: 67-г хайх (олдоно)



Зураг 1. Цайвар хөх — одоогийн хүрээ [lo, hi], хар хөх — шалгаж буй items[mid], саарал — хаягдсан хэсэг. 16 элементээс 4 эргэлтэд олдлоо. Linear search бол 11 харьцуулалт хийх байсан.

Эргэлт	lo	hi	mid	items[mid]	Харьцуулалт	Үйлдэл
1	0	15	7	41	41 < 67	баруун тийш: `lo = 8`
2	8	15	11	72	72 > 67	зүүн тийш: `hi = 10`
3	8	10	9	61	61 < 67	баруун тийш: `lo = 10`
4	10	10	10	67	**67 == 67**	`return 10`

Тохиолдол 2: 40-ийг хайх (байхгүй)



Зураг 2. Сүүлийн эргэлтийн дараа lo = 7, hi = 6 болж бие биеэ «гаталсан» тул хүрээ хоосон. Анхаар: lo нь 40-ийг оруулах байрлалыг яг зааж байна (5-р бүлэг).

Эргэлт	lo	hi	mid	items[mid]	Харьцуулалт	Үйлдэл
1	0	15	7	41	41 > 40	зүүн тийш: `hi = 6`
2	0	6	3	12	12 < 40	баруун тийш: `lo = 4`
3	4	6	5	23	23 < 40	баруун тийш: `lo = 6`
4	6	6	6	38	38 < 40	баруун тийш: `lo = 7`
—	7	6			`lo > hi`	`return -1`

Зөвлөгөө: компьютерээр мөрдүүлэх

Давталтын эхэнд нэг мөр `print` нэмбэл хүснэгтийг Python өөрөө хэвлэнэ. Код буруу ажиллаж байвал хамгийн түрүүнд үүнийг хий:

```
while lo <= hi:
    mid = (lo + hi) // 2
    print(f"lo={lo:2}  hi={hi:2}  mid={mid:2}  items[mid]={items[mid]}")
    ...
```

04 Түгээмэл алдаа ба засвар

Binary search ердөө 10 мөр боловч алдаа гаргахад маш амархан. Жон Бентли «Programming Pearls» номдоо мэргэжлийн програмистуудын ихэнх нь үүнийг анх удаа алдаагүй бичиж чадаагүйг тэмдэглэсэн байдаг. Доорх 5 алдааг бүгдийг нь бодитоор ажиллуулж шалгав.

Алдаа 1 · Давталтын нөхцөлд = мартах

БУРУУ

```
while lo < hi:
```

ЗӨВ

```
while lo <= hi:
```

Юу болох вэ: `binary_search([5], 5)` → **−1**. `binary_search([1, 3], 3)` → **−1**.

Яагаад: `lo == hi` үед 1 элемент үлдсэн байхад давталт түүнийг шалгалгүй гарна.

Алдаа 2 · Хүрээг багасгахгүй байх

БУРУУ

```
lo = mid
```

ЗӨВ

```
lo = mid + 1
```

Юу болох вэ: `binary_search([1, 3], 3)` → програм **гацна** (мөнхийн давталт).

Яагаад: lo = 0, hi = 1 → mid = 0 → 1 < 3 → lo = 0. Хүрээ өөрчлөгдөхгүй тул ижил эргэлт дахин дахин давтагдана. `hi = mid` гэвэл мөн адил.

Алдаа 3 · hi-г уртаар эхлүүлэх

БУРУУ

```
lo, hi = 0, len(items)
```

ЗӨВ

```
lo, hi = 0, len(items) - 1
```

Юу болох вэ: `binary_search([1, 3, 5], 9)` → **IndexError: list index out of range**.

Яагаад: 3 элементтэй жагсаалтын сүүлийн индекс 2. hi = 3 үед `items[3]`-д хандахыг оролдоно.

Алдаа 4 · Энгийн хуваалт хэрэглэх

БУРУУ

```
mid = (lo + hi) / 2
```

ЗӨВ

```
mid = (lo + hi) // 2
```

Юу болох вэ: `binary_search([1, 3, 5], 3)` → **TypeError: list indices must be integers or slices, not float**.

Яагаад: Python 3-т `/` үргэлж бутархай (`1.0`) буцаадаг. Индекс бүхэл байх ёстой.

Алдаа 5 · Эрэмбэгүй жагсаалтад хэрэглэх

БУРУУ

```
binary_search([9, 2, 7, 4, 1], 9)
```

ЗӨВ

```
items.sort()
binary_search(items, 9)
```

Юу болох вэ: Алдаа **гарахгүй**, гэхдээ **−1** буцаана — 9 индекс 0-д байгаа ч!

Яагаад: Дунд элемент 7 < 9 тул «9 баруун талд» гэж дүгнээд зүүн хагасыг хаясан. Энэ бол хамгийн аюултай алдаа: програм унахгүй, зүгээр л буруу хариу өгнө. Эрэмбэлэх боломжгүй бол linear search хэрэглэ.

Python-д байхгүй, бусад хэлэнд байдаг алдаа

Java, C++ зэрэгт `lo + hi` нь бүхэл тооны хязгаараас хэтэрч (overflow) сөрөг болж болдог тул `mid = lo + (hi - lo) // 2` гэж бичдэг. Python-ий бүхэл тоо хязгааргүй тул энд санаа зовох хэрэггүй, гэхдээ бусад хэлэнд шилжихдээ санаарай.

05 Давхардал, оруулах байр, bisect

Давхардсан утга: аль индексийг буцаах вэ?

`binary_search([1, 3, 3, 3, 5, 8], 3)` нь **2** буцаана — 3 гэсэн утга мөн, гэхдээ **эхний** 3 (индекс 1) биш. Үндсэн хувилбар «аль нэг тохирох индексийг» л олдог. Эхнийхийг олохын тулд **2 мөр** өөрчилнө: олдмогц зогсохгүй, санаж аваад зүүн тийш үргэлжлүүлэн хайна:

```
def first_occurrence(items, target):
    lo, hi = 0, len(items) - 1
    result = -1
    while lo <= hi:
        mid = (lo + hi) // 2
        if items[mid] == target:
            result = mid                # ← олдлоо: санаж аваад
            hi = mid - 1                # ← зүүн талд дахиад байгаа эсэхийг хайна
        elif items[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return result

first_occurrence([1, 3, 3, 3, 5, 8], 3)     # 1
```

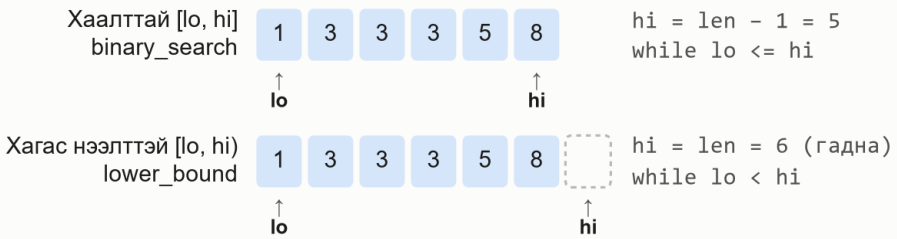
Сүүлийн тохиолдлыг олохын тулд `hi = mid - 1`-ийн оронд `lo = mid + 1` гэж бичнэ (8-р бүлгийн дасгал). Тэгвэл **хэдэн удаа** байгааг `last - first + 1` гэж O(log n)-д олно — `count()` бол O(n).

Оруулах байр (lower bound)

3-р бүлгийн 2-р тохиолдолд давталт дуусахад **lo нь 40-ийг оруулах байрыг** зааж байсан. Үүнийг тусдаа функц болгоё: «target-ээс бага биш (≥) эхний элементийн индекс». Энэ нь давхардалтай үед эхний тохиолдлыг, байхгүй үед оруулах байрыг өгнө:

```
def lower_bound(items, target):
    lo, hi = 0, len(items)              # ← hi = len: «жагсаалтын гадна» гэсэн байр
    while lo < hi:                      # ← хүрээ [lo, hi) хоосон биш байхад
        mid = (lo + hi) // 2
        if items[mid] < target:
            lo = mid + 1
        else:
            hi = mid                    # ← mid өөрөө хариу байж болох тул хасахгүй
    return lo

nums = [1, 3, 3, 3, 5, 8]
lower_bound(nums, 3)    # 1  — эхний 3
lower_bound(nums, 4)    # 4  — 4-ийг 5-ын өмнө оруулна
lower_bound(nums, 9)    # 6  — хамгийн төгсгөлд
```



Зураг 3. Хоёр загварын эхлэх төлөв. Хагас нээлттэй загварт hi нь жагсаалтын гаднах «хоосон» байрыг заадаг — тэнд оруулах боломжтой учраас.

Хоёр загварыг бүү холь

Энд `hi = len(items)`, `while lo < hi`, `hi = mid` гэж бичсэн — эдгээр нь 4-р бүлэгт «алдаа» байсан! Гэвч энд хүрээг **[lo, hi)** — hi-г **оруулахгүйгээр** тодорхойлсон тул бүгд зөв. Нэг загвар сонгоод **бүх мөрийг** түүнд тааруул:

	Хаалттай [lo, hi]	Хагас нээлттэй [lo, hi)
Эхлэл	`hi = len(items) - 1`	`hi = len(items)`
Давталт	`while lo <= hi`	`while lo < hi`
Зүүн тийш	`hi = mid - 1`	`hi = mid`
Баруун тийш	`lo = mid + 1`	`lo = mid + 1`

Python-ий bisect модуль

Python-д хагас нээлттэй загвар бэлэн байдаг. Бодит төсөлд үүнийг хэрэглэ — шалгагдсан бөгөөд C хэл дээр бичигдсэн тул хурдан.

```
import bisect

nums = [1, 3, 3, 3, 5, 8]
bisect.bisect_left(nums, 3)     # 1  ← манай lower_bound-тай яг ижил
bisect.bisect_right(nums, 3)    # 4  ← 3-аас ИХ эхний элементийн индекс
bisect.bisect_right(nums, 3) - bisect.bisect_left(nums, 3)    # 3 → 3 гурван удаа байна

bisect.insort(nums, 4)          # эрэмбийг эвдэлгүй оруулна
print(nums)                     # [1, 3, 3, 3, 4, 5, 8]
```

`bisect` ашиглан бидний гэрээтэй (индекс эсвэл −1) хайлт бичих нь:

```
from bisect import bisect_left

def binary_search(items, target):
    i = bisect_left(items, target)              # оруулах байр
    if i < len(items) and items[i] == target:   # тэр байранд target өөрөө байна уу?
        return i
    return -1
```

Анхаар: `insort` нь байрыг O(log n)-д олдог ч бусад элементийг шилжүүлж оруулдаг тул нийтдээ **O(n)**.

Хариуг хайх: жагсаалтгүй binary search

Binary search-д жагсаалт заавал биш. «`mid * mid <= n` үнэн үү?» гэсэн асуулт 0, 1, 2, 3, … дээр **үнэн, үнэн, …, үнэн, худал, худал, …** гэж зөвхөн **нэг удаа** солигддог. Ийм (монотон) асуултын «солигдох цэг»-ийг binary search-ээр олно:

```
def isqrt(n):
    """k * k <= n байх хамгийн их бүхэл k (бүхэл квадрат язгуур)."""
    lo, hi = 0, n
    ans = 0
    while lo <= hi:
        mid = (lo + hi) // 2
        if mid * mid <= n:
            ans = mid                  # mid тохирно — илүү томыг хайя
            lo = mid + 1
        else:
            hi = mid - 1
    return ans

isqrt(50)           # 7   (7 * 7 = 49 <= 50,  8 * 8 = 64 > 50)
isqrt(1_000_000)    # 1000 — ердөө 20 орчим алхамд
```

06 Тестлэх

«Ажиллаж байх шиг байна» гэж орхиж болохгүй — 4-р бүлгийн алдаанууд ихэнх жишээн дээр зөв хариу өгдөг! Хоёр төрлийн тест бичье.

1. Захын тохиолдлууд (edge cases)

Алдаа ихэвчлэн хил дээр нуугддаг: хоосон, ганц элемент, эхний ба сүүлийн байр.

```
def test_edges():
    assert binary_search([], 5) == -1            # хоосон жагсаалт
    assert binary_search([5], 5) == 0            # ганц элемент, байгаа
    assert binary_search([5], 3) == -1           # ганц элемент, байхгүй
    assert binary_search([1, 3, 5, 7], 1) == 0   # эхний элемент
    assert binary_search([1, 3, 5, 7], 7) == 3   # сүүлийн элемент
    assert binary_search([1, 3, 5, 7], 0) == -1  # бүгдээс бага
    assert binary_search([1, 3, 5, 7], 9) == -1  # бүгдээс их
    assert binary_search([1, 3, 5, 7], 4) == -1  # хооронд нь, байхгүй
    assert binary_search([1, 3, 5], 5) == 2      # сондгой урттай
    print("Захын тохиолдлууд: OK")
```

`assert` нь нөхцөл худал бол `AssertionError` шиднэ, үнэн бол юу ч хийхгүй.

2. Санамсаргүй тест: linear search-тэй харьцуулах

Linear search маш энгийн тул бараг алдаагүй. Тиймээс түүнийг «зөв хариу» болгон ашиглаж, мянга мянган санамсаргүй жагсаалт дээр binary search-тэй харьцуулна:

```
import random

def test_random(trials=10_000):
    for _ in range(trials):
        n = random.randint(0, 20)
        items = sorted(random.sample(range(50), n))   # эрэмбэлэгдсэн, давхардалгүй
        target = random.randint(-5, 55)               # заримдаа байхгүй утга
        assert binary_search(items, target) == linear_search(items, target), \
            (items, target)                           # бүтэлгүйтвэл жишээг хэвлэнэ
    print(trials, "санамсаргүй тест: OK")
```

Яагаад **давхардалгүй**? Давхардалтай үед хоёр функц өөр өөр (гэхдээ хоёулаа зөв) индекс буцааж болно — жишээ нь 3-ыг хайхад linear нь 1, binary нь 2. `random.sample` давхардалгүй утга сонгодог.

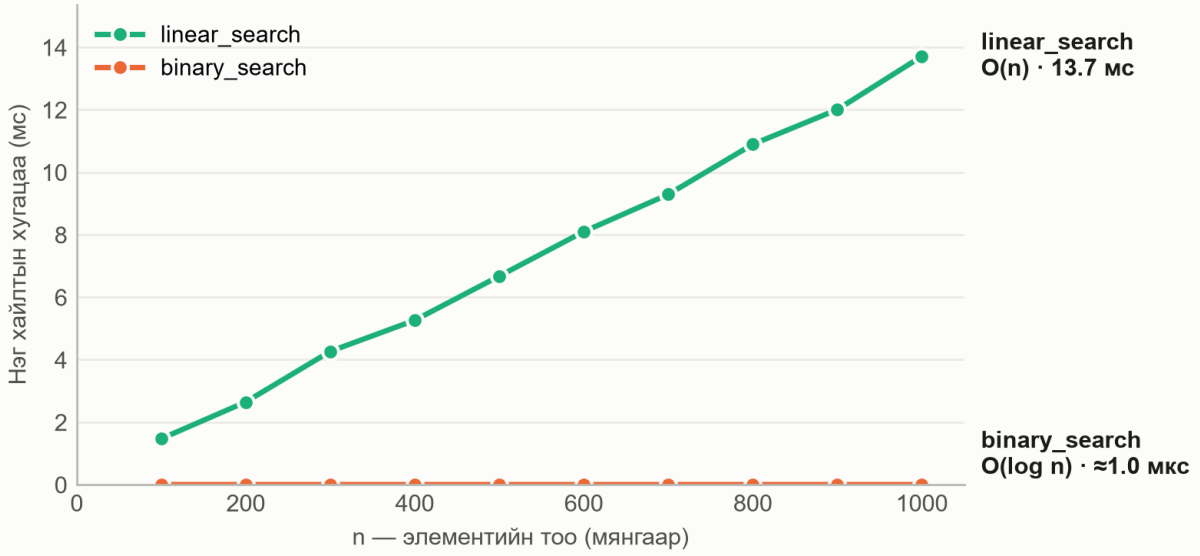
Энэ PDF-ийг үүсгэх үед зөв хувилбарыг **10 000** санамсаргүй тестээр шалгахад бүгд амжилттай давлаа. Харин 4-р бүлгийн алдаатай хувилбаруудыг ижил тестэд оруулахад:

Хувилбар	1 000 тестээс бүтэлгүйтсэн	Анхны алдаа	Алдааны төрөл
Алдаа 1: `while lo < hi`	**72**	5-р тест	буруу хариу
Алдаа 2: `lo = mid`	**679**	1-р тест	гацсан
Алдаа 3: `hi = len(items)`	**235**	1-р тест	IndexError
Алдаа 4: `mid = (lo + hi) / 2`	**955**	1-р тест	TypeError
Зөв `binary_search`	**0**	—	—

> **Дүгнэлт**
> Захын тохиолдлын тест + энгийн хувилбартай харьцуулах санамсаргүй тест = хэрэгжүүлэлтдээ итгэх хамгийн хялбар арга. Алдаа бүр эхний 5 тестийн дотор илэрсэн.

07 Linear search-тэй харьцуулах

Хоёр функцийг 100 000 … 1 000 000 элементтэй жагсаалт дээр **байхгүй** утга хайлгаж (хамгийн муу тохиолдол) хэмжив:



Зураг 4. Энэ PDF-ийг үүсгэх үед бодитоор хэмжсэн үр дүн (хэд хэдэн давталтын хамгийн бага утга). Таны компьютер дээр тоо нь өөр гарах ч хэлбэр нь ижил.

n = 1 000 000 үед linear search **13.7 мс**, binary search **1.0 микросекунд** — ойролцоогоор **14 000 дахин** элгаатай. Binary search-ийн шугам бараг хэвтээ: сая элементэд ч ердөө 20 харьцуулалт хийдэг.

Бүрэн харьцуулалт

	Linear search	Binary search
Урьдчилсан нөхцөл	Байхгүй	**Эрэмбэлэгдсэн** байх
Цаг (хамгийн муу)	O(n)	O(log n)
Санах ой	O(1)	O(1) давталттай, O(log n) рекурсив
Өгөгдлийн бүтэц	Ямар ч дараалал: list, файлын мөр, генератор	Индексээр шууд хандах боломжтой (list)
Хайх нөхцөл	Дурын нөхцөл (тэгш тоо, 60-аас бага…)	Эрэмбэтэй уялдсан нөхцөл (≥ x, ≤ x…)
Код	4 мөр, алдаа гаргахад хэцүү	10 мөр, ±1 алдаа түгээмэл

Эрэмбэлээд хайх нь ашигтай юу?

Эрэмбэлэх (`sort()`) өөрөө **O(n log n)** — нэг linear search-ээс илүү үнэтэй. Тэгэхээр хэдэн удаа хайхаас шалтгаална. n = 1 000 000 үед (log n ≈ 20) нийт алхмыг ойролцоогоор тооцъё:

Хайлтын тоо (k)	Linear: k · n	Эрэмбэлэх + binary: n log n + k log n	Ялагч
1	1 000 000	20 000 020	**Linear**
10	10 000 000	20 000 200	**Linear**
100	100 000 000	20 002 000	**Эрэмбэлээд binary**
10 000	10 000 000 000	20 200 000	**Эрэмбэлээд binary**

Ойролцоогоор **20-оос олон** удаа хайх бол нэг удаа эрэмбэлээд binary search хэрэглэх нь ашигтай.

Аль нь вэ? Шийдвэрийн жагсаалт

1. **Жагсаалт жижиг** (хэдэн арав) эсвэл **ганцхан удаа** хайх уу? → **Linear**. Энгийн, хангалттай хурдан.
2. **Дурын нөхцөлөөр** хайх уу (эхний тэгш тоо, 60-аас бага оноо)? → **Linear**.
3. Өгөгдөл **эрэмбэлэгдсэн** эсвэл **олон удаа** хайх уу? → **Binary** (`bisect`).
4. «x-ээс их эхний», «a ба b хооронд хэд», «хамгийн ойр утга» гэх мэт асуулт уу? → **Binary** (`bisect_left`, `bisect_right`).
5. Зөвхөн «байгаа юу?» гэж асуух, эрэмбэ хамаагүй юу? → `set` эсвэл `dict`: дунджаар O(1) (Big O хичээлийн 5-р бүлэг).

08 Дасгал

Эхлээд өөрөө бодож, кодоо 6-р бүлгийн аргаар тестлээрэй. Хариуг дараагийн хуудаснаас шалга.

> **Дасгал 1 · Гараар мөрдөх**
>
> `binary_search([3, 8, 15, 21, 30, 42, 57], 30)`-ийг 3-р бүлгийн хүснэгтээр мөрд: эргэлт бүрийн `lo`, `hi`, `mid`, `items[mid]`. Хэдэн эргэлт хийж, юу буцаах вэ?

Дасгал 2 · Алдааг ол

Доорх кодоор `[1, 3]`-аас 0-ийг хайхад юу болох вэ? Яагаад? Хэрхэн засах вэ?

```
def binary_search(items, target):
    lo, hi = 0, len(items) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if items[mid] == target:
            return mid
        elif items[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return -1
```

Дасгал 3 · Сүүлийн тохиолдол

`last_occurrence(items, target)`-ийг O(log n)-д бич. Зааварчилгаа: 5-р бүлгийн `first_occurrence`-оос нэг мөр өөрчилнө.

```
last_occurrence([1, 3, 3, 3, 5, 8], 3)   # 3
```

Дасгал 4 · Хүрээнд хэд байна?

Эрэмбэлэгдсэн жагсаалтад `a <= x <= b` байх утга хэд байгааг O(log n)-д тоол. `bisect` ашиглаж болно.

```
count_in_range([1, 3, 3, 5, 8, 13], 3, 8)   # 4   (3, 3, 5, 8)
```

Дасгал 5 · Хамгийн ойр утга

Эрэмбэлэгдсэн, хоосон биш жагсаалтаас `x`-д хамгийн ойр утгыг буцаа. Тэнцүү зайтай бол багыг нь. Зааварчилгаа: `bisect_left`-ийн өгсөн байр ба түүний өмнөх элементийг харьцуул.

```
closest([2, 5, 9, 14], 11)   # 9   (11 - 9 = 2,  14 - 11 = 3)
```

Дасгалын хариу

Бүх хариуг энэ PDF-ийг үүсгэх үед жишээн дээр нь ажиллуулж шалгасан.

1. Гараар мөрдөх — 3 эргэлт, 4 буцаана

Эргэлт	lo	hi	mid	items[mid]	Харьцуулалт	Үйлдэл
1	0	6	3	21	21 < 30	`lo = 4`
2	4	6	5	42	42 > 30	`hi = 4`
3	4	4	4	30	**30 == 30**	`return 4`

2. Алдааг ол — мөнхийн давталт

lo = 0, hi = 1 → mid = 0, 1 > 0 → hi = 0. Дараа нь lo = 0, hi = 0 → mid = 0, 1 > 0 → hi = 0 … хүрээ өөрчлөгдөхгүй тул програм гацна. Засвар: `hi = mid - 1` (хаалттай загварт `while lo <= hi`-тэй хамт заавал ±1).

3. Сүүлийн тохиолдол — O(log n)

```python
def last_occurrence(items, target):
    lo, hi = 0, len(items) - 1
    result = -1
    while lo <= hi:
        mid = (lo + hi) // 2
        if items[mid] == target:
            result = mid
            lo = mid + 1            # ← баруун талд дахиад байгаа эсэхийг хайна
        elif items[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return result
```

4. Хүрээнд хэд байна — O(log n)

```python
from bisect import bisect_left, bisect_right

def count_in_range(items, a, b):
    # b-ээс их эхний байр  −  a-аас бага биш эхний байр
    return bisect_right(items, b) - bisect_left(items, a)
```

5. Хамгийн ойр утга — O(log n)

```python
from bisect import bisect_left

def closest(items, x):
    i = bisect_left(items, x)           # x-ээс бага биш эхний байр
    if i == 0:
        return items[0]                 # x бүгдээс бага (эсвэл тэнцүү)
    if i == len(items):
        return items[-1]                # x бүгдээс их
    before, after = items[i - 1], items[i]
    return before if x - before <= after - x else after
```

09 Хураангуй

Хоёр загвар — цээжлэх биш, ойлгож бич

■ Хаалттай [lo, hi] · хайлт

```python
def binary_search(items, target):
    lo, hi = 0, len(items) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if items[mid] == target:
            return mid
        elif items[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Олдвол индекс, олдохгүй бол −1.
Сая элементэд 20 харьцуулалт.

■ Хагас нээлттэй [lo, hi) · байр

```python
def lower_bound(items, target):
    lo, hi = 0, len(items)
    while lo < hi:
        mid = (lo + hi) // 2
        if items[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

target-ээс бага биш эхний байр.
= bisect.bisect_left

Binary search бичихдээ шалгах 5 зүйл

1. Жагсаалт **эрэмбэлэгдсэн** үү?
2. `hi = len(items) - 1` ба `while lo <= hi` — хоёулаа **хаалттай** [lo, hi] загвар уу?
3. `mid = (lo + hi) // 2` — **бүхэл** хуваалт уу?
4. `lo = mid + 1`, `hi = mid - 1` — хүрээ эргэлт бүрт **заавал** багасаж байна уу?
5. Захын тохиолдлуудаар (хоосон, 1 элемент, эхний, сүүлийн, байхгүй) **тестэлсэн** үү?

Санаж үлдэх зүйлс

- Гол санаа: target байгаа бол заавал `items[lo..hi]` дотор; хүрээ хоосон болбол — байхгүй.
- Эргэлт бүр хүрээг **хагас** болгодог тул **O(log n)**: n 1000 дахин → +10 алхам.
- Кодоо **хүснэгтээр мөрд**; ойлгомжгүй бол давталт дотор print нэм.
- Давхардал, оруулах байр, хүрээнд тоолох → `bisect_left` / `bisect_right`.
- Шинэ хэрэгжүүлэлтээ **linear search-тэй** санамсаргүй тестээр харьцуул.
- Эрэмбэгүй, жижиг, нөхцөлөөр хайх бол — linear search хангалттай.